

SSpMV: A Sparsity-aware SpMV Framework Empowered by Multimodal Machine Learning

Shengle Lin^a, Chubo Liu^{a*}, Yan Ding^a, Joey Tianyi Zhou^{b,c}, Kenli Li^a and Wangdong Yang^{a*}

^a College of Computer Science and Electronic Engineering, Hunan University, China

^b Centre for Frontier AI Research, Agency for Science, Technology and Research, Singapore

^c Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore
{lsl036, liuchubo, ding, lkl, yangwangdong}@hnu.edu.cn, zhouty@cfar.a-star.edu.sg

Abstract—Sparse Matrix-Vector Multiplication (SpMV) is an essential sparse operation in scientific computing and artificial intelligence. Efficiently adapting SpMV algorithms to diverse matrices and architectures requires a framework capable of accurately recognizing sparse patterns and selecting the optimal implementation. In this work, we introduce Sparsity-aware SpMV (SSpMV), a framework that integrates expert-designed features with multimodal representations to adaptively predict the best-performing algorithm and parameters. For this purpose, we design a multimodal neural network called MM-Adapter, to capture diverse modalities to represent the computational features of SpMV. Experimental results demonstrate that MM-Adapter achieves the highest accuracy of 81.05%, outperforming existing SpMV prediction models. Furthermore, SSpMV consistently delivers substantial performance improvements over state-of-the-art sparse libraries across various multi-core platforms.

Index Terms—Sparse Matrix, SpMV, Machine Learning.

I. INTRODUCTION

With advancements in hardware and the exponential growth of data, sparse linear algebra has emerged as a cornerstone in circuit simulation, electronic design automation, and artificial intelligence [1]–[3]. A fundamental operation within sparse computing paradigms is Sparse Matrix-Vector Multiplication (SpMV), defined as $y = Ax$, which plays a pivotal role in solving sparse linear systems [4]. The SpMV kernel is a critical component of iterative solvers such as Generalized Minimal Residual (GMRES) and Preconditioned Conjugate Gradient (PCG), which involve repeated operations on matrices with fixed sparse structures [5], [6]. Therefore, optimizing the performance of SpMV across diverse platforms is essential due to its significant computational demands.

Achieving consistently high-performance SpMV kernels on multi-core platforms remains challenging due to irregular memory access patterns and limited utilization of vector instructions. As listed in Table I, previous approaches often introduce specialized formats, such as SELL- c - σ [7], CSR5 [8], BCSR [9], tailored to specific hardware architectures to enhance computational efficiency. However, as illustrated in Fig. 1, no single format optimally suits all matrices, as performance depends on specific sparse patterns. Some recent studies prefer to leverage machine learning models, such as decision

TABLE I
FORMATS WITH HYPERPARAMETERS USED IN SSpMV

Format	Method	Params	Description
COO	OpenMP, Lb	SCH	SCH is the OpenMP scheduling mode, including Static (St), StaticChunk (StC), Dynamic (Dyn) and Guided (Gd). Lb denotes parallelism with a load-balanced partition by nnz.
CSR	OpenMP, Lb	SCH	
DIA	OpenMP, Lb	SCH	
ELL	OpenMP, Lb	SCH	
S-ELL	S-ELL (Lb)	SCH	c is the row chunk size that affects vector paddings. σ defines the reordering blocks of rows to reduce fillings while trying to keep locality.
	SELL- c - σ	c, σ	
	SELL- c -R	SCH, c	
BCSR	OpenMP, Lb	SCH, bR, bC	bR, bC control the block size by SIMD width and alignment.
CSR5	SIMD Intrinsics	σ_5, ω	ω is the size of SIMD unit. σ_5 is defined by cache capacity.

trees (DT) [10], [11], support vector machines (SVM) [12], multilayer perceptrons (MLP) [13], and convolutional neural networks (CNN) [14], to adaptively select the best format for specific matrices. Alphasparse [15] automatically creates machine-designed GPU kernels based on the input pattern. While such frameworks consider many expert-designed matrix features, **the intricate interplay between sparse patterns and architectural characteristics, such as memory locality and vectorization, remains underexplored**. Furthermore, many studies rely on limited or synthetic datasets, which diminishes their applicability to real-world sparse systems, such as those found in circuit simulation problems.

To this end, we propose a Sparsity-aware SpMV framework (SSpMV) to address these issues. Specifically, SSpMV consists of two core components: 1) a suite of SpMV algorithms deeply optimized across different multi-core CPUs; and 2) an intelligent adapter, MM-Adapter, which employs a comprehensive set of multimodal representations to map a sparse pattern to its most efficient SpMV algorithm. Moreover, we utilize finite element software FreeFEM [16] and computational fluid dynamics software OpenFoam [17] to generate an extensive dataset of matrices, including a wide range of real-world scenarios. Our experimental results on over 6,000 sparse matrices reveal that SSpMV achieves an average speedup of $3.08\times$, with a maximum of $71.7\times$, compared to the Intel MKL [18]. The main contributions of this work are as follows:

* Corresponding authors.

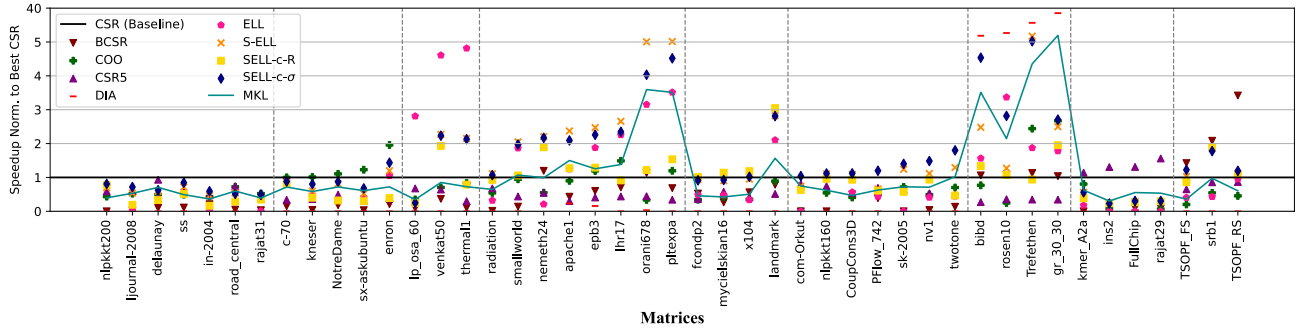


Fig. 1. Speedup of double-precision SpMV in different sparse formats and Intel MKL SpMV kernel compared to the best implementation of CSR format.

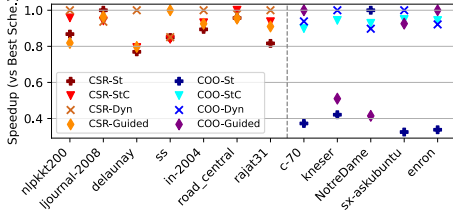
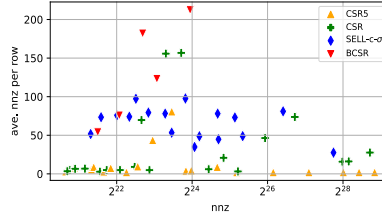
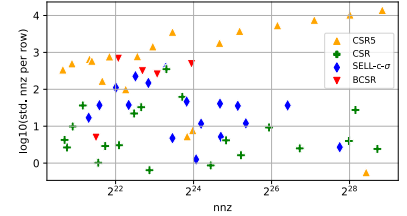


Fig. 2. Performance degradation of CSR and COO by different scheduling methods.



(a) Average number of nnz



(b) Standard deviation of nnz

Fig. 3. Distribution of fastest method when concerning features of row nnz .

- (1) We develop a high-performance Sparsity-aware SpMV framework that automatically maps a sparse matrix to its optimal SpMV implementation.
- (2) We design a multimodal convolutional neural network, MM-Adapter, capable of extracting diverse modalities of a sparse pattern to represent the architectural features relevant to SpMV computation.
- (3) We conduct extensive experiments on a large dataset of real-world sparse systems, demonstrating significant performance improvements achieved by SSpMV.

II. BACKGROUND AND MOTIVATION

The SpMV operation ($y = Ax$) multiplies a sparse matrix A by a dense vector x , resulting in a dense output vector y . A wide variety of sparse storage formats have been developed to balance compression efficiency and SpMV performance. To comprehensively evaluate these algorithms and uncover the potential of various formats and parameters, we conduct extensive benchmarks on matrices from SuiteSparse Collection [19]. All methods listed in Table I are evaluated, where *COO*, *CSR*, *DIA* and *ELL* represent the four fundamental storage formats; *BCSR* [9], *S-ELL Series* [7] and *CSR5* [8] are vector-friendly optimized formats. The performance distributions reveal the following key insights:

1. The four basic formats still have their own performance advantages: As illustrated in Fig.1, each SpMV format excels with specific matrices, even for the four basic formats. Specifically, COO and CSR are well-suited for the sparse structures with irregular or fragmented nonzeros. In contrast, ELL outperforms S-ELL Series when row lengths

are uniform, because balanced nonzero distributions allow ELL to achieve good memory efficiency without additional segmentation or reordering. DIA exhibits significant speedup for main-diagonalized nonzero patterns, with accelerations ranging from $11.4\times$ to $34.8\times$ in examples. In addition, the performance of *Intel MKL dcscr_SpMV* is recorded, revealing that it is suboptimal for the majority of matrices.

2. Picking the suitable parameters is important for SpMV performance: Fig.2 shows the significant impact of different OpenMP scheduling policies (*St*, *StC*, *Dyn* and *Guided*) on SpMV performance. The performance degradation, measured relative to the optimal scheduling strategy, reveals that CSR fluctuates between $0.77\times$ and $1\times$, while COO can suffer up to a $3.1\times$ slowdown. This disparity arises because the COO parallelizes based on individual nonzeros, making it highly sensitive to scheduling policies. Given the variability of sparse patterns, identifying a universally optimal scheduling strategy is inherently challenging, let alone tuning additional configuration parameters.

3. Existing expert-designed features are insufficient to characterize sparse patterns: We consider two widely used features about the number of non-zeros (nnz) per row: the average and the standard deviation [10], [11], [20]. From Fig.3a, we observe that BCSR and SELL- $c-\sigma$ perform better on datasets with higher row nnz , while CSR and CSR5 generally favor more sparser patterns. Fig.3b reveals a cleaner distribution pattern for the CSR5 format, showing its marked advantage in cases with extreme imbalance. However, when the workload decreases and nnz becomes similar between rows, it is difficult to predict which method will be the fastest.

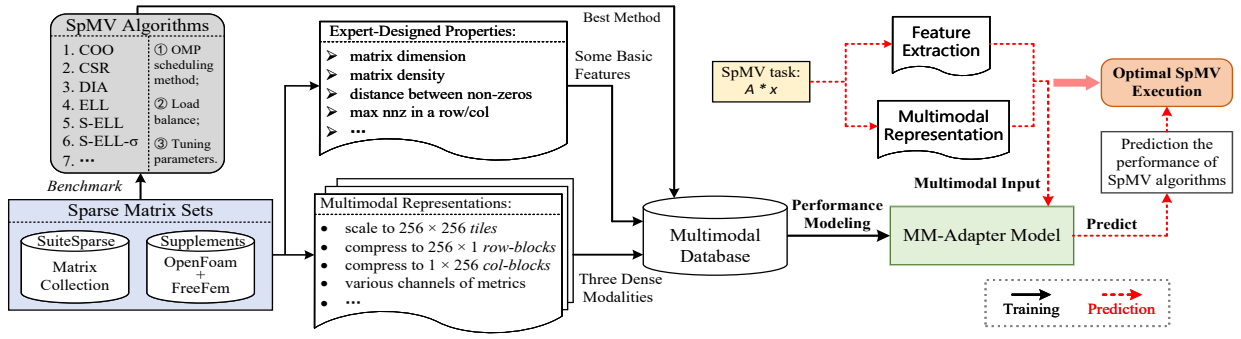


Fig. 4. Overview of the Sparsity-aware SpMV framework.

Granularity	Features	Description
Matrix	row, col, nnz, density	Depict the overall Matrix
	nnz_ratio_on_main_diag	Capture the distribution of nnz on diagonals
	ave nnz distance	Capture the locality to the input vector
Rows	$nz_R, max_R, \mu_R, \sigma_R, G_R, P_R$	Determine the row scheduling method and paddings
Columns	$nz_C, max_C, \mu_C, \sigma_C, G_C, P_C$	Determine the memory access pattern to input vector
Tiles	$nz_T, max_T, \mu_T, \sigma_T, G_T, P_T$	Describe the balance of nnz across Tiles
	uniqR, uniqC	Describe the locality inside a Tile
	reuseR, reuseC	Describe the data reuses probability across Tiles
Row Block	$nz_{RB}, max_{RB}, \mu_{RB}, \sigma_{RB}, G_{RB}, P_{RB}$	Determine cache utilization and vectorization
Col Block	$nz_{CB}, max_{CB}, \mu_{CB}, \sigma_{CB}, G_{CB}, P_{CB}$	Determine cache utilization and cache-line alignment

The right figure shows a 5x5 grid representing a sparse matrix. It highlights different granularities for feature extraction: 'Columns' (entire columns), 'Column Block' (groups of columns), 'Rows' (entire rows), 'Tiles' (individual elements), 'Row Block' (groups of rows), and 'Col Block' (groups of columns). The grid is color-coded to show these different levels of abstraction.

Fig. 5. Expert-designed features and their descriptions. The right figure shows the different granularities for feature extraction.

Overall, a comprehensive exploration of all formats and parameters enables the SpMV kernel to exploit the performance potential, but the intricate relationship between sparse patterns and architectural features makes it challenging to tell optimal implementation based solely on expert-designed features.

III. SPARSITY-AWARE SPMV FRAMEWORK

A. Overview of SSpMV

Given the above observation and analysis, we propose SSpMV, a Sparsity-aware SpMV framework that selects the most efficient format and implementation for a given sparse matrix. Fig.4 provides an overview of SSpMV. First, the framework generates kernels for all formats listed in Table I based on hardware characteristics such as the length of SIMD units and cache line size. For each input matrix, SSpMV extracts expert-designed features and multimodal representations to capture the architectural features relevant to SpMV computation. Through benchmarking on extensive datasets, it constructs a feature-label pair for each matrix. To enhance sparsity awareness, we develop a multimodal machine learning model, named *MM-Adapter*, which leverages the multimodal features of matrix sparsity to predict the most suitable SpMV implementation. As a result, SSpMV ensures optimal selection among various formats and parameters, maximizing SpMV performance for different sparse matrices across platforms.

B. Expert-designed Features

As a sparsity-aware framework, SSpMV begins with a feature extraction process to collect a broad range of expert-designed features. Some features directly characterize the overall matrix, such as matrix size, density, the proportion of

nnz on the main diagonals, and the average distance. Other carefully selected features can reflect memory access patterns, vector paddings, and data skew, forming the basis for accurate performance modeling of SpMV operation.

Fig.5 summarizes all the expert-designed features used in SSpMV. Specifically, we partition the sparse matrix at different granularities to compute the six fine-grained features from each perspective. In addition to rows and columns, we partition the sparse matrix into 256×256 tiles, 256×1 row blocks, and 1×256 column blocks, respectively. Each partition is referred to as a "bucket", and the following six fine-grained expert-designed features are calculated:

- 1) nz : Ratio of buckets that are not zero;
- 2) max : Maximum of nnz between buckets;
- 3) μ : Mean of nnz between buckets;
- 4) σ : Standard deviation of nnz between buckets;
- 5) G : The *Gini coefficient* of the nnz among buckets ($0 \sim 1$);
- 6) P : The p -ratio of the nnz among buckets. P -ratio means that $p\%$ of the buckets have $(1-p)\%$ nonzeros ($0 \sim 0.5$).

The first four features are commonly used to characterize the statistical properties of data distributions [9], [11], [21]. The last two features measure equilibrium: in a perfectly balanced scenario, the p -ratio is 0.5 and the Gini coefficient is 0. As described in Fig.5, these 40 features provide a preliminary assessment of the sparse structures.

C. Multimodal Representations

Representing the matrix sparsity typically encounters two significant challenges. First, sparse matrices vary greatly in scale, making it difficult to effectively apply fixed-resolution convolutional neural networks (CNNs). Second, the complex

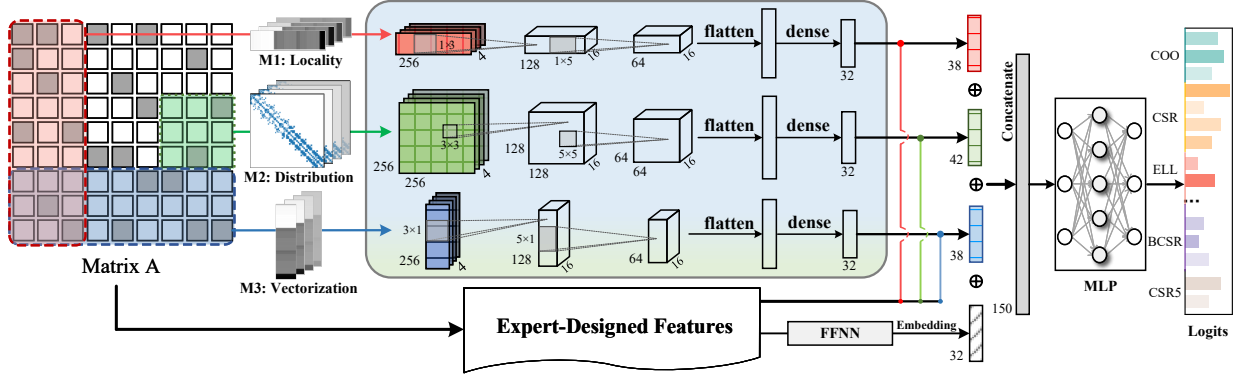


Fig. 6. Details of MM-Adapter. Multimodal representations are extracted by CNNs and fused with expert-designed features to model SpMV performance.

sparsity of matrices results in unique preferences for memory access patterns, scheduling strategies, and other architectural factors, which cannot be fully captured by existing approaches [14], [21]. Inspired by multimodal machine learning, we introduce novel multimodal representations that comprehensively encapsulate the diverse properties of sparse structures.

The unique challenges in SpMV have led us to propose a new interpretation of multimodality in the context of SpMV operation. Unlike traditional multimodal learning that deals with different data types, our approach focuses on characterizing the diverse architectural features relevant to sparse matrix computation. Specifically, we introduce three distinct modalities that reflect hardware preferences: **Locality**, **Distribution**, and **Vectorization**, each addressing specific requirements of SpMV kernels. We obtain these three types of multimodal representations by compressing the sparse matrices with granularity aligned to the scale of expert feature extraction. As shown in Fig.6, the **Locality** modality (M_1 , red block) reflects the access pattern to the input vector, reflecting memory locality during computation. The **Distribution** modality (M_2 , green tile) captures a snapshot of the workload within the matrix, providing insights for load balancing strategies. Finally, the **Vectorization** modality (M_3 , blue block) represents latent features of a sparse matrix when row vectorization is applied.

Indeed, the compression of each modal representation may result in the loss of structural details. To mitigate this issue, we design different channels for each modality to encode statistical information related to original sparse patterns:

- C_1 : Nnz of each modality;
- C_2 : Maximum nnz per row of each modality;
- C_3 : Average nnz per row of each modality;
- C_4 : Standard deviation of nnz per row of each modality.

The multi-channel multimodal representations enable SSpMV to comprehensively perceive matrix sparsity and effectively explore architectural properties critical to SpMV computation.

D. MM-Adapter Design

After extracting the three architecture-related modalities, we employ a multimodal machine learning model for feature fusion. Given the impressive success of CNNs in the image

domain and the ability of multimodal representations to overcome matrix size variability, we select CNNs as the backbone of our *Multimodal Adapter* (MM-Adapter).

As illustrated in Fig.6, MM-Adapter requires four distinct inputs derived from the sparse matrix A : three architectural modality representations, and one expert-designed feature set. For the modalities associated with locality and vectorization, CNNs employ a series of *conv1D* layers to capture global structures, while for the distribution modality, *conv2D* layers are used to extract local patterns. Subsequently, the expert-designed features, which serve as **the fourth artificial modality**, are integrated as an augmentation to the latent features, aligned with the specific partitioning granularity of each modality. A feed-forward neural network (FFNN) then processes all artificial features and transforms them into a dense embedding. Finally, a multilayer perceptron (MLP) takes the concatenated embedding as a fused input and maps the given matrix to the optimal SpMV implementation.

To construct the matrix labels, we compute the selection probability for each SpMV algorithm as follows:

$$P_i = \frac{G_i}{G_1 + G_2 + \dots + G_N}, \quad (1)$$

where G_i represents the GFlops achieved by the i -th algorithm. The optimal SpMV kernel is identified as the algorithm with the highest probability, as shown in the "Logits" of Fig.6. With this training methodology, the MM-Adapter can be easily adapted to any platform, and new kernels can be integrated into SSpMV to enhance its diversity.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

1) Machine: We evaluate SSpMV on two multi-core architectures: an x86_64 platform equipped with two *Intel Xeon Gold 5120* processors, providing a total of 28 CPU cores, and an aarch64 platform with two *Kunpeng 920* processors, offering a total of 128 CPU cores. The MM-Adapter is trained on an NVIDIA-A100 (40GB) GPU.

2) Dataset: In addition to the 2,834 real matrices from the SuiteSparse Collection, we generate 3,218 supplementary

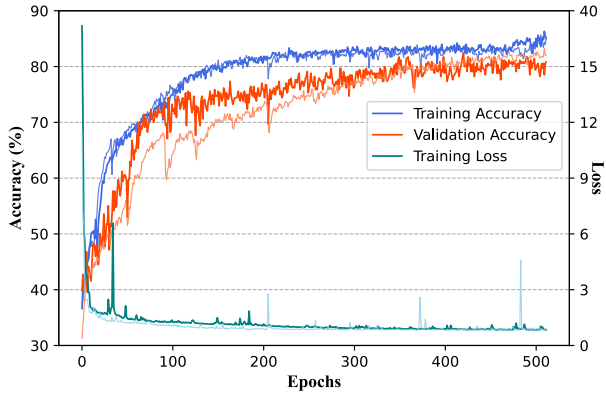


Fig. 7. Loss and accuracy curve of the MM-Adapter.

matrices by sample programs from FreeFEM and OpenFoam, totaling 6,052 matrices. The sample programs include the Poisson equation, Laplace3D equation, Circuit Equation, and other scientific scenarios. The combined dataset is split into 70% for training, 10% for validation, and 20% for testing.

3) Implementations: For the source code of various SpMV formats and implementations, we utilize *C++17* and *OpenMP* for shared-memory parallel programming. All algorithms utilize the OpenMP *simd* pragma for automatic vectorization and employ explicit load-balancing optimization strategies.

As summarized in Table I, we use all four scheduling methods for COO and CSR, while DIA utilizes only the *St* scheduling policy. For the S-ELL (ELL) formats, both *StC* and *Dyn* are considered, with chunk size c strictly matching the vector width of the computing platform. The reordering factor σ is chosen from $\{2^9, 2^{14}\}$, aligning with the maximum lengths typically buffered in the L1 and L2 caches, respectively. For BCSR and CSR5, we set $b_R = \sigma_5$ to the cache-line width and $b_C = \omega$ to the vector length, using only the *StC* strategy for chunk parallelism. In total, we obtain 21 candidate implementations for all SpMV formats and parameters.

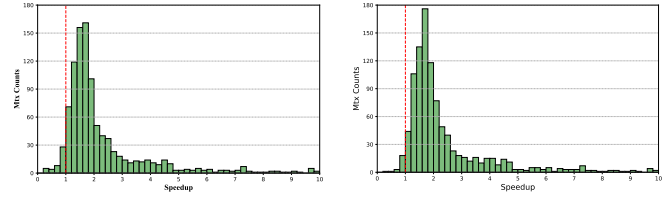
B. MM-Adapter Evaluation

Training the MM-Adapter shown in Fig.6 with 4,236 matrices takes approximately 37 minutes offline. Fig.7 presents the accuracy and loss curves during training and validation, with transparency lines indicating variability over multiple experiments. As training data accumulate, the MM-Adapter progressively refines its modeling accuracy. After 500 epochs, the average training and validation accuracies reach 85.08% and 82.06%, respectively, while the final loss stabilizes around 0.3. The results demonstrate that the MM-Adapter effectively captures the multimodal architectural features relevant to SpMV computation, requiring only a few epochs to converge.

Table II presents the runtime results for 1,210 matrices in the test set and compares MM-Adapter with several existing SpMV prediction models. The results demonstrate significant improvements by MM-Adapter in all three evaluation metrics, achieving 62.63% precision, 59.41% recall, and 81.05% accuracy, respectively. In contrast, the inferior performance of

TABLE II
PREDICTION EVALUATION AND ABLATION STUDY.

Framework	Precision(%)	Recall(%)	Accuracy(%)
SMAT [10]	15.92	13.24	30.69
DIESEL [20]	46.72	48.30	62.77
SpNet [9]	41.21	32.92	75.88
MatNet [21]	52.86	46.76	72.90
MM-Adapter	62.63	59.41	81.05
MM-Adapter Ablation Study			
only channel C1	56.67	51.05	75.94
w/o MLP	60.55	50.47	79.90
w/o Modality 1	56.85	54.72	75.35
w/o Modality 1&3	56.18	51.08	72.63
w/o Modality 2	36.48	31.54	70.15
w/o Expert Features	57.19	51.47	73.51



(a) Predicted algorithms by SSpMV (b) Hand-pick best SpMV algorithms

Fig. 8. Speedup Distribution of isolated SpMV performance over Intel MKL.

SMAT (Decision Tree), DIESEL (MLP), SpNet and MatNet (CNN) underscores their limited generalizability to this multi-classification task, mainly due to their inability to capture sufficient architectural features about SpMV computation.

Moreover, an ablation study is conducted to examine the architecture of MM-Adapter, with results provided in the lower part of Table II. The findings highlight the importance of each component: 1) Different channels enhance the MM-Adapter's capabilities, facilitating the extraction of implicit information in each modality. 2) The MLP module helps the model to be more comprehensive in searching for positive examples, even though its contribution to accuracy is marginal. 3) **M2: Distribution** is identified as the most crucial modality, as it contains the richest structural features, while M1&M3 are similar in importance to the artificial modality.

C. Speedup Distribution

Using the 1,210 matrices in the test set, we evaluate the overall speedup distribution of SSpMV on the Intel Xeon 5120 platform. The SpMV routine from *Intel MKL* is selected as a baseline. As shown in Fig.8a, the acceleration of SSpMV ranges from a minimum of $0.15\times$ to a maximum of $71.7\times$, with an average speedup of $3.08\times$. Fig.8b presents the results of hand-picking the best SpMV implementation, where the speedup ranges from $0.21\times$ to $71.7\times$, with an average of $3.19\times$. The two figures show similar distributions, with the main difference being the migration of datasets from the $[0,1]$ interval to the $[1,2]$ interval. In addition, some datasets in the $[1,2]$ range failed to achieve optimal acceleration, primarily due to a mismatch in the predicted hyperparameters. In conclusion, with a mere $0.11\times$ performance gap, SSpMV approaches the theoretical optimal algorithm very closely.

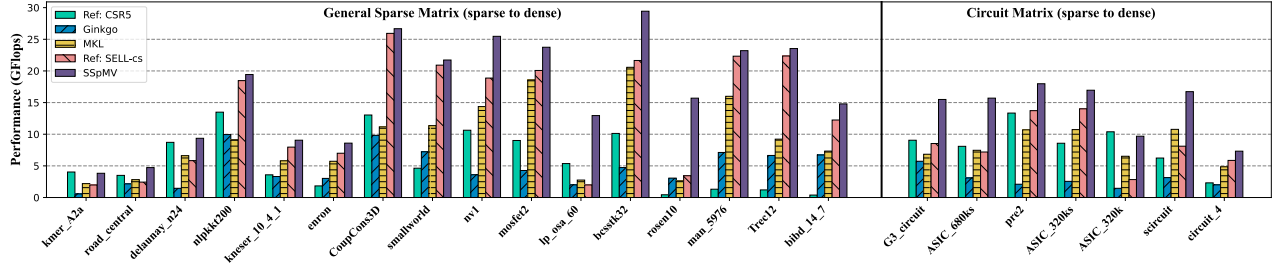


Fig. 9. Performance comparison with other libraries on Intel Xeon 5120 processors.

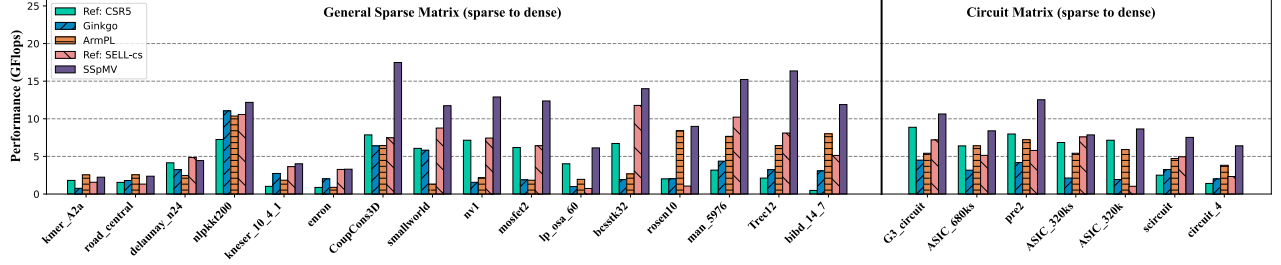


Fig. 10. Performance comparison with other libraries on Huawei Kunpeng 920 processors.

D. Acceleration for Matrix Cases

To assess the performance acceleration of SSpmv, we compare it against vector-friendly formats and well-established sparse linear algebra libraries, including CSR5 [8], SELL-c- σ [7], *Ginkgo* [22], *Intel MKL* 24.2.0 [18] on x86_64, and *ArmPL* 24.10 [23] on aarch64. In our experiments, CSR5 and SELL-c- σ use their default configuration parameters; while *Ginkgo*, *MKL*, and *ArmPL* are executed by their default executor routines. The SSpmv framework also considers the inference overhead, with an average of 3.19 milliseconds per matrix case. Fig.9 and Fig.10 display double precision SpMV performance results for 16 general sparse matrices and 7 circuit matrices from the SuiteSparse Collection. It is worth noting that circuit matrices represent highly specialized cases due to their irregular distribution of non-zero elements [2].

The results on both platforms show that CSR5 outperforms other formats on extremely sparse matrices due to its compressed tile structure which offers better vector efficiency. In general, *Intel MKL* exhibits the most stable performance across varying matrix types on the x86_64 platform, while both *Ginkgo* and *ArmPL* show significant performance fluctuations on both platforms. This performance gap primarily arises from the structural variations in different sparse patterns, which pose significant challenges in maintaining consistent high performance for SpMV. Fortunately, our SSpmv effectively overcomes this limitation, providing stable and high-performance SpMV across tested matrices. Although inference overhead occasionally impacts speedup, the experimental results demonstrate that SSpmv enhances the adaptability of the SpMV implementations to different sparse patterns.

Overall, on Intel Xeon 5120, compared with CSR5, SELL-c- σ , *Ginkgo* and *Intel MKL*, SSpmv obtains an average speed

of $6.70\times$, $1.85\times$, $4.63\times$ and $2.07\times$, respectively. Similarly, on Huawei Kunpeng 920, SSpmv delivers an average speedup of $3.50\times$, $2.51\times$, $3.54\times$ and $2.68\times$ over CSR5, SELL-c- σ , *Ginkgo*, and *ArmPL*. In particular, for matrices from Circuit Simulation Problems, SSpmv can achieve state-of-the-art performance compared with other libraries.

V. CONCLUSION

In this paper, we implement a suite of high-performance SpMV algorithms tailored to various widely used formats and introduce a Sparsity-aware SpMV framework, SSpmv, which adaptively selects the optimal format and parameters for a given sparse matrix. By leveraging expert-designed features combined with innovative multimodal representations, our MM-Adapter achieves state-of-the-art prediction accuracy. The experimental results show that SSpmv can achieve an average acceleration of $3.08\times$, closely approximating the theoretical optimum of $3.19\times$. Moreover, the matrix case study reveals that SSpmv can deliver significant performance benefits for different matrices across multi-core platforms, demonstrating its effectiveness and portability.

ACKNOWLEDGMENT

We deeply appreciate all the reviewers for their invaluable comments. We also thank Hantao Xiong and Yang He for their significant contributions to this work. This work was partially supported by the Major Projects of Xiangjiang Laboratory (22XJ01011), the Key Program of National Natural Science Foundation of China (U21A20461, U24A20235), the National Natural Science Foundation of China (Grant Nos. 62321003 and 62227808), China Postdoctoral Science Foundation (2024M750879, BX20240111) and Changsha Natural Science Foundation (kq2502277).

REFERENCES

- [1] J. Zhao, Y. Wen, Y. Luo, Z. Jin, W. Liu, and Z. Zhou, “Sflu: Synchronization-free sparse lu factorization for fast circuit simulation on gpus,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 37–42.
- [2] T. Wang, W. Li, H. Pei, Y. Sun, Z. Jin, and W. Liu, “Accelerating sparse lu factorization with density-aware adaptive matrix multiplication for circuit simulation,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [3] D. Garyfallou, N. Evmorfopoulos, and G. Stamoulis, “Large scale circuit simulation exploiting combinatorial multigrid on massively parallel architectures,” in *2018 7th International Conference on Modern Circuits and Systems Technologies (MOCAST)*, 2018, pp. 1–4.
- [4] F. Sadi, J. Sweeney, T. M. Low, J. C. Hoe, L. Pileggi, and F. Franchetti, “Efficient spmv operation for large and highly sparse matrices using scalable multi-way merge parallelization,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 347–358.
- [5] Y. Saad and M. H. Schultz, “Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems,” *SIAM Journal on scientific and statistical computing*, vol. 7, no. 3, pp. 856–869, 1986.
- [6] E. F. Kaasschieter, “Preconditioned conjugate gradients for solving singular systems,” *Journal of Computational and Applied mathematics*, vol. 24, no. 1-2, pp. 265–275, 1988.
- [7] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop, “A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide simd units,” *SIAM Journal on Scientific Computing*, vol. 36, no. 5, pp. C401–C423, 2014.
- [8] W. Liu and B. Vinter, “Csr5: An efficient storage format for cross-platform sparse matrix-vector multiplication,” in *Proceedings of the 29th ACM on International Conference on Supercomputing*, 2015, pp. 339–350.
- [9] J. C. Pichel and B. Pateiro-Lopez, “Sparse matrix classification on imbalanced datasets using convolutional neural networks,” *IEEE Access*, vol. 7, pp. 82 377–82 389, 2019.
- [10] J. Li, G. Tan, M. Chen, and N. Sun, “Smat: An input adaptive auto-tuner for sparse matrix-vector multiplication,” in *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*, 2013, pp. 117–126.
- [11] S. Yesil, A. Heidarshenas, A. Morrison, and J. Torrellas, “Wise: Predicting the performance of sparse matrix vector multiplication with machine learning,” in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2023, pp. 329–341.
- [12] S. Chen, J. Fang, C. Xu, and Z. Wang, “Adaptive hybrid storage format for sparse matrix-vector multiplication on multi-core simd cpus,” *Applied Sciences*, vol. 12, no. 19, p. 9812, 2022.
- [13] A. Benatia, W. Ji, Y. Wang, and F. Shi, “Machine learning approach for the predicting performance of spmv on gpu,” in *2016 IEEE 22nd International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2016, pp. 894–901.
- [14] Y. Zhao, J. Li, C. Liao, and X. Shen, “Bridging the gap between deep learning and sparse matrix format selection,” in *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*, 2018, pp. 94–108.
- [15] Z. Du, J. Li, Y. Wang, X. Li, G. Tan, and N. Sun, “Alphasparse: Generating high performance spmv codes directly from sparse matrices,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [16] F. Hecht, “New development in freefem++,” *Journal of numerical mathematics*, vol. 20, no. 3-4, pp. 251–266, 2012.
- [17] H. Jasak, A. Jemcov, Z. Tukovic *et al.*, “Openfoam: A c++ library for complex physics simulations,” in *International workshop on coupled methods in numerical dynamics*, vol. 1000, 2007, pp. 1–20.
- [18] E. Wang, Q. Zhang, B. Shen, G. Zhang, X. Lu, Q. Wu, and Y. Wang, “Intel math kernel library,” *High-Performance Computing on the Intel® Xeon Phi™: How to Fully Exploit MIC Architectures*, pp. 167–188, 2014.
- [19] T. A. Davis and Y. Hu, “The university of florida sparse matrix collection,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 38, no. 1, pp. 1–25, 2011.
- [20] T. Mohammed, A. Albesri, I. Katib, and R. Mehmood, “Diesel: A novel deep learning-based tool for spmv computations and solving sparse linear equation systems,” *The Journal of Supercomputing*, vol. 77, no. 6, pp. 6313–6355, 2021.
- [21] Z. Xie, G. Tan, W. Liu, and N. Sun, “Ia-spgemm: An input-aware auto-tuning framework for parallel sparse matrix-matrix multiplication,” in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 94–105.
- [22] H. Anzt, T. Cojean, G. Flegar, F. Göbel, T. Grützmacher, P. Nayak, T. Ribizel, Y. M. Tsai, and E. S. Quintana-Ortí, “Ginkgo: A Modern Linear Operator Algebra Framework for High Performance Computing,” *ACM Transactions on Mathematical Software*, vol. 48, no. 1, pp. 2:1–2:33, Feb. 2022. [Online]. Available: <https://doi.org/10.1145/3480935>
- [23] Arm, “Arm performance libraries,” <https://developer.arm.com/tools-and-software/server-and-hpc/compile/arm-compiler-for-linux/arm-performance-libraries>.